MANU2478 – Advanced Robotics

Practical Assignment

Team #2
Minh Duc Vu – s4054648
Nguyen Vu Tran Khoi – s3978262

May 2025

School/Department/Area of Science, Engineering, and Technology

Table of Contents

1. Question 1 3
1.1. DH parameters of the robot 3
1.2. Jacobian matrix of the robot 7
1.3. Trajectory tracking 8
1.4. MATLAB simulation 12
Question 2 18
2.1. DEMO TASK: Palletising 18
2.1.1. Palletising Task Summary – RMIT Education Cell 18
2.1.2. System Hardware Components 18
2.2. GOFA Robot Writing Operation 27
4. References 34

1. Question 1

1.1. DH parameters of the robot

The coordinates of each joint in the robot are already defined in. SLDPRT files, so measure tool in SolidWorks is used to derive the translation for coordinates of one joint to another to progressively get the result shown in table 1.

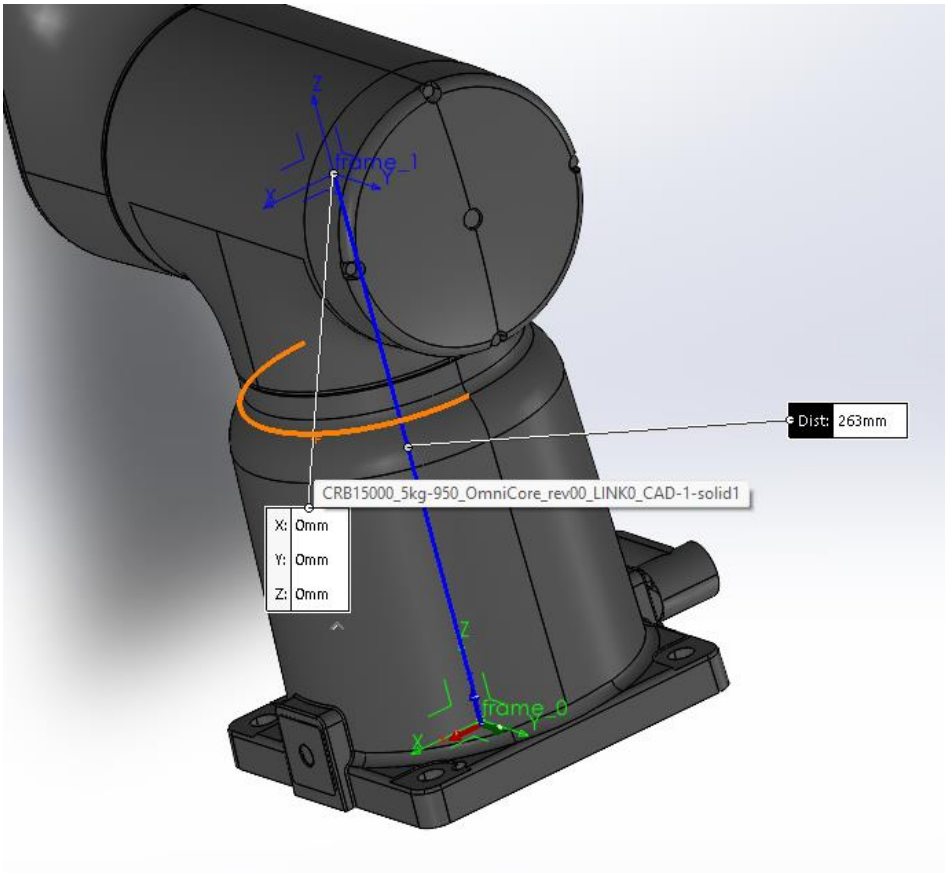


Figure 1: Frame measurement

For example, based on the measurement done via SolidWorks as shown in the figure above, from frame 0 to frame 1, the angle from Z0 to Z1 about X0 is 0deg, the distance from Z0 to Z1 measured along X0 is 0mm, the distance from X0 to X1 measured along Z1 is 263mm and the angle form X0 to X1 measured about Z1 is 0deg. The procedure goes the same for frame 1 to frame 2 and so on to frame 6 to end effector. Finally, the DH parameters table is as below.

Table 1: DH parameters

i	alpha{i-1}	a{i-1}	d{i}	theta{i}
1	0	0	263	0
2	90	0	0	90
3	0	444	0	0
4	90	110	470	0
5	-90	0	0	0
6	90	79.94	101	0
EF	90	0	152.72	90

Then this function is used to describe the position of one joint with respect to the previous one

$$ {}^{i-1}_{i}T = \begin{bmatrix} c\theta_i & -s\theta_i & 0 & a_{i-1} \\ s\theta_i c\alpha_{i-1} & c\theta_i c\alpha_{i-1} & -s\alpha_{i-1} & -s\alpha_{i-1}d_i \\ s\theta_i s\alpha_{i-1} & c\theta_i s\alpha_{i-1} & c\alpha_{i-1} & c\alpha_{i-1}d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}. $$

Figure 2: Translation matrix

The translation matrix above can also be written in MATLAB script as 'DH_parameter' function below

```
function T = DH_transform(link_length,link_twist,link_offset,joint_value)
a= link_length;
alpha = link_twist;
d= link_offset;
theta = joint_value;

T = sym(zeros(4,4));

T(1,1) = cosd(theta);
T(1,2) = -sind(theta);
T(1,3) = 0;
T(1,4) = a;

T(2,1) = sind(theta)*cosd(alpha);
T(2,2) = cosd(theta)*cosd(alpha);
T(2,3) = -sind(alpha);
T(2,4) = -sind(alpha)*d;

T(3,1) = sind(theta)*sind(alpha);
T(3,2) = cosd(theta)*sind(alpha);
T(3,3) = cosd(alpha);
T(3,4) = cosd(alpha)*d;

T(4,1) = 0;
T(4,2) = 0;
T(4,3) = 0;
T(4,4) = 1;
```

Figure 3: Translation matrix MATLAB script

Example usage: By plugging in the DH parameters of joint 1 (Table 1.1) into the 'DH_transform' function, the result is:

```
T01 = DH_transform(0,0,263,theta1);
```

Figure 4: Example of using DH parameters function

This is the description of position of joint 1 with respect to joint 0 (or the base frame). Combine the table and translation formulation, the description of one frame with respect to the previous one is as follows

```
syms theta1 theta2 theta3 theta4 theta5 theta6 % in deg
T01 = DH_transform(0,0,263,theta1);
T12 = DH_transform(0,90,0,theta2);
T23 = DH_transform(444,0,0,theta3);
T34 = DH_transform(110,90,470,theta4);
T45 = DH_transform(0,-90,0,theta5);
T56 = DH_transform(79.94,90,101,theta6);
T6EF = DH_transform(0,90,152.72,90);
```

Figure 5: Full implementation of DH_transform

Notice that for ‘T6EF’ the ‘theta’ is a fix value (90 degrees) while for the other translations, this value is not constant, this is because ‘T6EF’ is the offset of the end effector from joint 6, since end effector does not ‘rotate’ or ‘move’ with respect to joint 6 so this translation has to be constant. The value 90 degrees is also derived from measurement in SolidWorks. During testing, for T6EF, the matrix is as follows

```
T6EF =

[0, -1,  0,        0]
[0,  0, -1, -3818/25]
[1,  0,  0,        0]
[0,  0,  0,        1]
```

Figure 6: Translation of end effector with respect to joint 6

But from the actual measurement from Solidwork, there is one extra offset in X axis that the DH parameter does not cover (the red line - 93.5mm).

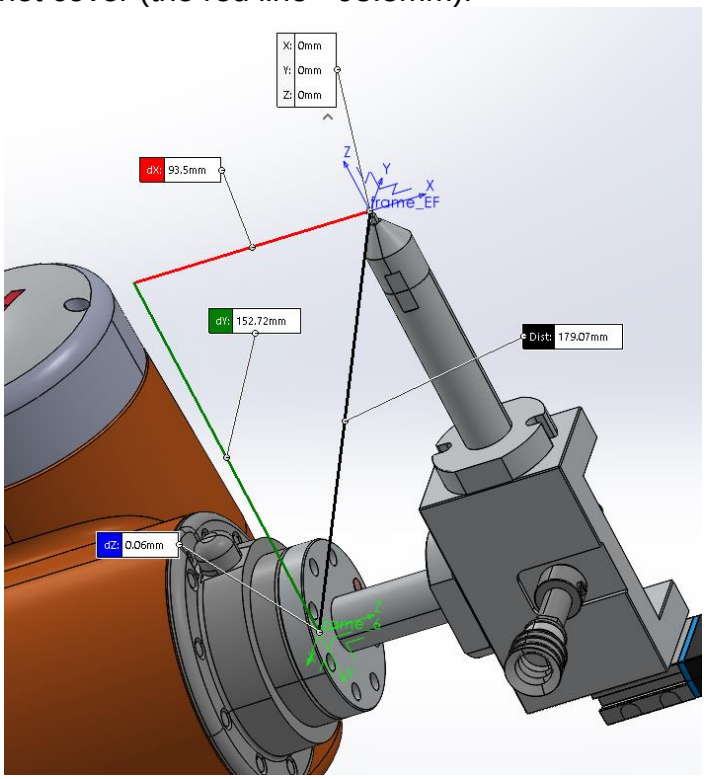


Figure 6: End Effector Offset SolidWorks CAD

Therefore, extra offset of 93.5mm is needed, which can be written in matrix form as

```
T_offset = [1 0 0 93.5;
           0 1 0 0;
           0 0 1 0;
           0 0 0 1];
```

Figure 7: Offset for end effector

Finally, the description of the end effector with respect to joint 6 is as follows

```
T0_EF = T01*T12*T23*T34*T45*T56*T6EF*T_offset;
```

Figure 8: Translation of end effector to base frame

To verify the correctness of the DH parameters, perform a test using the initial joint angles as follows.

```
T_test = double(subs(T_final,[t1 t2 t3 t4 t5 t6 ],[0 pi/2 0 0 0 0]));
```

Figure 9: Testing transformation matrix

Which means theta1, theta2, theta3. theta4. theta5, theta6 is 0, pi/2, 0, 0, 0, 0 respectively (the original angles derived from table 1.1).
These angles are expected to make the robot have the same pose compare with the original pose in the SolidWorks file.

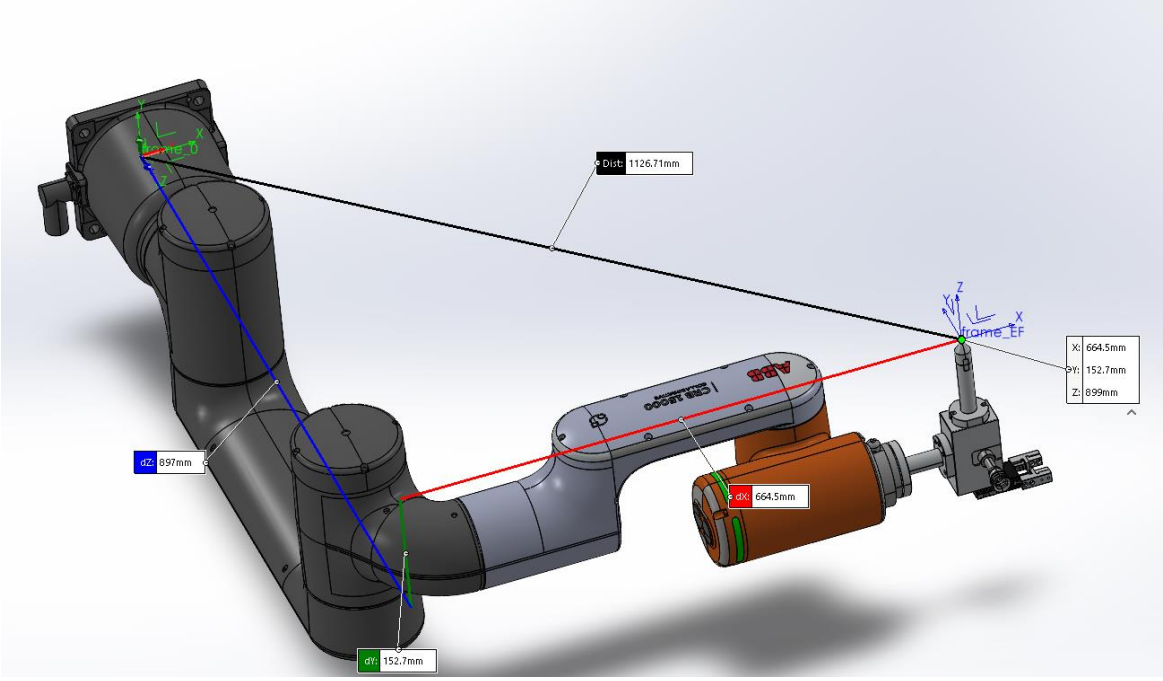


Figure 10: Original pose of GOFA5 robot

The end effector now has orientation as shown in the matrix below which perfectly matches the original pose.

```
T_test =

    1.0000         0         0  664.5000
         0         0    1.0000  152.7200
         0   -1.0000         0  896.9400
         0         0         0    1.0000

>>
```

Figure 11: Test translation of end effector with respect to frame 0

Therefore, the DH parameters and the translation matrix is corrected.

1.2. Jacobian matrix of the robot

From the previous task, the end effector has been successfully described with respect to the base frame using the translation matrix 'T0_EF'. The linear position of the EF relative to the base frame is given by T0_EF[1:3, 4].

```
p_ef_x = T0_EF(1,4);
p_ef_y = T0_EF(2,4);
p_ef_z = T0_EF(3,4);
```

Figure 12: Position of end effector with respect to base frame

Then the linear Jacobian is derived based on the x, y, z position of EF from above using the formula

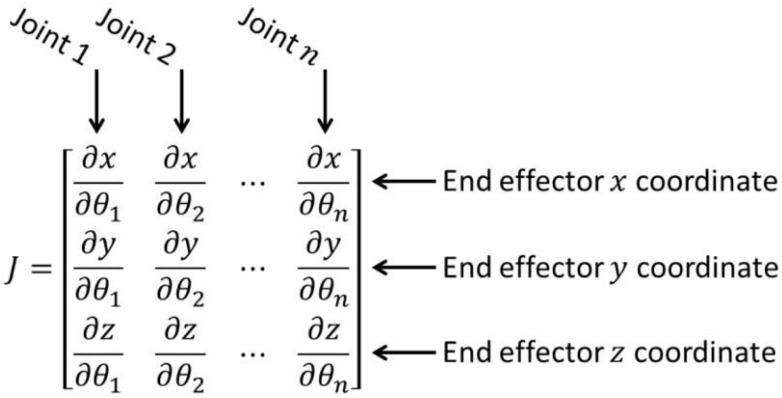


Figure 13: Formula to derive linear Jacobian

Which can also be written as

```
% Linear Jacobian
 J11 = simplify(diff(p_ef_x_short,t1));
 J12 = simplify(diff(p_ef_x_short,t2));
 J13 = simplify(diff(p_ef_x_short,t3));
 J14 = simplify(diff(p_ef_x_short,t4));
 J15 = simplify(diff(p_ef_x_short,t5));
 J16 = simplify(diff(p_ef_x_short,t6));

 J21 = simplify(diff(p_ef_y_short,t1));
 J22 = simplify(diff(p_ef_y_short,t2));
 J23 = simplify(diff(p_ef_y_short,t3));
 J24 = simplify(diff(p_ef_y_short,t4));
 J25 = simplify(diff(p_ef_y_short,t5));
 J26 = simplify(diff(p_ef_y_short,t6));

 J31 = simplify(diff(p_ef_z_short,t1));
 J32 = simplify(diff(p_ef_z_short,t2));
 J33 = simplify(diff(p_ef_z_short,t3));
 J34 = simplify(diff(p_ef_z_short,t4));
 J35 = simplify(diff(p_ef_z_short,t5));
 J36 = simplify(diff(p_ef_z_short,t6));
```

Figure 14: MATLAB script for linear Jacobian matrix

Where 'diff(f, x)' is the derivative of 'f' with respect to 'x', so we got the linear Jacobian (first 3 rows of the Jacobian matrix).
The angular Jacobian (last 3 rows of the Jacobian matrix) is derived from this formula.

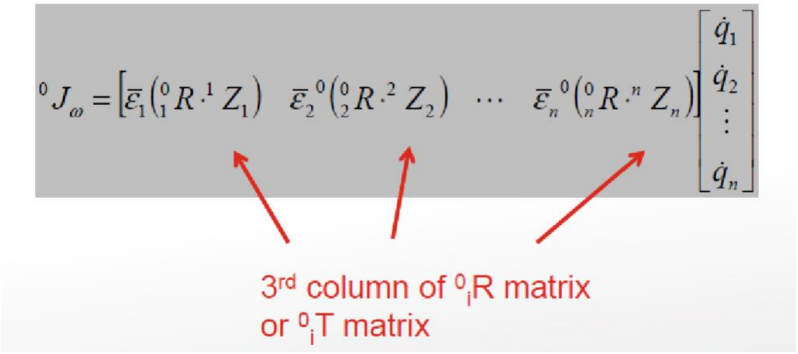


Figure 15: Formula to derive angular Jacobian

Since all joints are revolute so all $\bar{\epsilon} = 1$, so basically the orientation of z axis of all joints with respect to base frame, here is the code that demonstrate the workflow

```
% Angular Jacobian
% Third column of rotation matrix or the orientation of z axis of that
% frame with respect to base joint / frame
T01_short = subs(T01,[theta1 theta2 theta3 theta4 theta5 theta6], [t1*180/pi t2*180/pi t3*180/pi t4*180/pi t5*180/pi t6*180/pi]);
J41 = T01_short(1,3);
J51 = T01_short(2,3);
J61 = T01_short(3,3);

T02 = T01*T12;
T02_short = subs(T02,[theta1 theta2 theta3 theta4 theta5 theta6], [t1*180/pi t2*180/pi t3*180/pi t4*180/pi t5*180/pi t6*180/pi]);
J42 = simplify(T02_short(1,3));
J52 = simplify(T02_short(2,3));
J62 = simplify(T02_short(3,3));

T03 = T02*T23;
T03_short = subs(T03,[theta1 theta2 theta3 theta4 theta5 theta6], [t1*180/pi t2*180/pi t3*180/pi t4*180/pi t5*180/pi t6*180/pi]);
J43 = simplify(T03_short(1,3));
J53 = simplify(T03_short(2,3));
J63 = simplify(T03_short(3,3));
```

Figure 16: MATLAB script to derive angular Jacobian

The same procedure is used to get J[4:6,4:6]. Finally, the full Jacobian matrix is written as

```
% Complete Jacobian matrix
J = [J11 J12 J13 J14 J15 J16;
     J21 J22 J23 J24 J25 J26;
     J31 J32 J33 J34 J35 J36;
     J41 J42 J43 J44 J45 J46;
     J51 J52 J53 J54 J55 J56;
     J61 J62 J63 J64 J65 J66];
```

Figure 17: Full implementation of Jacobian matrix

1.3. Trajectory tracking

First, the function: 'load('trajectory_data.mat')' loads the trajectory data. Then there is a new variable called 'lem_trajectory' shows up in a workspace.

Data overview:

lem_trajectory ×

2001×4 double

	1	2	3	4
1	0	0.0000	-0.0000	0
2	0.0100	0.0056	-0.0112	0
3	0.0200	0.0080	-0.0158	0
4	0.0300	0.0098	-0.0193	0
5	0.0400	0.0113	-0.0223	0
6	0.0500	0.0126	-0.0249	0
7	0.0600	0.0139	-0.0272	0
8	0.0700	0.0150	-0.0293	0

Figure 18: Lemniscate trajectory data

The trajectory data stored as a table with 2001 rows and 4 columns, first column represents time instant, column 2 to 4 are x, y and z coordinates of the end effector respectively, so now store x, y and z and time instant t as

```
% Extract trajectory data
x = lem_trajectory(:, 2) * 1000;   % X positions
y = lem_trajectory(:, 3) * 1000;   % Y positions
z = lem_trajectory(:, 4) * 1000;   % Z positions
t = lem_trajectory(:, 1);   % Time
```

Figure 19: Store trajectory data

This trajectory was scaled appropriately to millimeters and used to define the target position of the end-effector in 3D space.
A 3D path (the lemniscate trajectory) is given for the end effector (EF) of the GOFA robot to follow. The path is defined in Cartesian space (x, y, z), while the robot is controlled through joint angles (q1 to q6). The objective is to calculate the joint angles over time so that the EF follows the given path.

Workflow:

A numerical integration approach was used to simulate the motion. At each time step, the joint velocities were computed using the pseudo-inverse of the Jacobian to solve the underdetermined system:

$$\dot{q} = J^{\dagger} V$$

Figure 20: Formulation to derive joint velocity at each time instant using pseudoinverse

Where $\boldsymbol{V}$ is the desired spatial velocity vector and J^{+} is the pseudo-inverse of the Jacobian matrix. The joint positions were then updated using Euler integration, and the result was stored in a matrix for all time steps.

Initialization:

First, determine the total number of data points to ensure the number of joint angles matches the number of trajectory points. Then, create a vector of time differences between successive samples for use in Euler integration.

```
num_steps = length(x);      % Number of points in trajectory
dt = diff(t);               % Time steps (assume non-uniform time steps)
```

Figure 21: Number of data points and time steps

The joint angles are initialized such that the EF frame aligns with the trajectory's centre, as required by the assignment instructions, and as illustrated in the Fig. 2 of the handout.

```
% Initialize joint trajectory matrix
q_traj = zeros(num_steps, 6);          % Store joint angles over time

q_traj(1, :) = [0, pi/3, -pi/4, 0, 5*pi/12, -pi/2];  % Initial joint angles
```

Figure 22: Initialize joint angles

The EF's linear velocities along X, Y and Z directions are estimated numerically using finite differences between consecutive positions.

```
% Compute linear velocities numerically
v_x = [0; diff(x)./dt];  % Approximate dx/dt
v_y = [0; diff(y)./dt];  % Approximate dy/dt
v_z = [0; diff(z)./dt];  % Approximate dz/dt
```

Figure 23: Linear velocity of the end effector at each time instant

The symbolic 6x6 Jacobian matrix (computed in Task 1.b) is converted into a MATLAB function, allowing it to be evaluated numerically at any joint configuration.

```
% Define the function to evaluate the Jacobian numerically
% J is the symbolic 6x6 Jacobian from the code (from file 'task1_ab')
J_func = matlabFunction(J, 'Vars', [t1, t2, t3, t4, t5, t6]);
```

Figure 24: Function to compute Jacobian matrix

Core algorithm:

This is the core of the motion rate control algorithm:

- For each time step:
  - **Get the current joint angles.**
  - **Evaluate the Jacobian matrix** numerically at that configuration.
  - **Form the desired velocity vector**, which includes only linear velocity since the orientation must remain constant.
  - **Solve for joint velocities** $\dot{q}$ using the pseudo-inverse of the Jacobian.
  - **Update joint angles** using Euler integration:

$$\dot{q}_{i+1} = q_i + q_i \Delta t$$

- This loop simulates the robot's motion in joint space to track the Cartesian trajectory.

```
% Numerical integration to compute joint positions
for i = 1:num_steps-1
    % Current joint angles (in radians, adjusted for your substitution)
    q_current = q_traj(i, :);

    % Evaluate the Jacobian at the current joint angles
    J_current = J_func(q_current(1), q_current(2), q_current(3), ...
                       q_current(4), q_current(5), q_current(6));

    % Desired end effector velocity (linear only, angular velocity = 0)
    V_desired = [v_x(i); v_y(i); v_z(i); 0; 0; 0];

    % Solve for joint velocities using pseudo-inverse
    % J * q_dot = V_desired
    q_dot = pinv(J_current) * V_desired;

    % Update joint angles using Euler integration
    q_traj(i+1, :) = q_current + q_dot' * dt(i);
end
```

Figure 25: Motion rate control

The result is then saved as a .xlsx file

```
% Create a table with joint positions
joint_table = table(t, q_traj(:,1), q_traj(:,2), q_traj(:,3), ...
                    q_traj(:,4), q_traj(:,5), q_traj(:,6), ...
                    'VariableNames', {'Time', 'q1', 'q2', 'q3', 'q4', 'q5', 'q6'});

% Display the table
disp(joint_table);

% Optionally, save the table to a file
writetable(joint_table, 'joint_trajectory.xlsx');
```

Figure 26: Store result in .xlsx file

This is how joint angles at each instant look like

A	B	C	D	E	F	G
			jointtrajectory			
Time	q1	q2	q3	q4	q5	q6
Number	Number	Number	Number	Number	Number	Number
0.3800	-0.1388	0.9960	-0.6842	-0.1457	1.2619	-1.5266
0.3900	-0.1403	0.9952	-0.6825	-0.1472	1.2611	-1.5260
0.4000	-0.1417	0.9944	-0.6809	-0.1487	1.2604	-1.5254
0.4100	-0.1430	0.9936	-0.6793	-0.1502	1.2597	-1.5249
0.4200	-0.1444	0.9928	-0.6778	-0.1517	1.2590	-1.5243
0.4300	-0.1457	0.9920	-0.6762	-0.1531	1.2583	-1.5238
0.4400	-0.1470	0.9912	-0.6747	-0.1545	1.2576	-1.5232
0.4500	-0.1482	0.9904	-0.6731	-0.1558	1.2569	-1.5227
0.4600	-0.1495	0.9897	-0.6716	-0.1571	1.2562	-1.5222
0.4700	-0.1507	0.9889	-0.6701	-0.1585	1.2555	-1.5217
0.4800	-0.1519	0.9882	-0.6686	-0.1597	1.2549	-1.5212
0.4900	-0.1530	0.9874	-0.6671	-0.1610	1.2542	-1.5207
0.5000	-0.1541	0.9867	-0.6656	-0.1622	1.2535	-1.5202
0.5100	-0.1552	0.9859	-0.6642	-0.1634	1.2529	-1.5197
0.5200	-0.1563	0.9852	-0.6627	-0.1646	1.2522	-1.5192
0.5300	-0.1574	0.9845	-0.6613	-0.1657	1.2516	-1.5187
0.5400	-0.1584	0.9837	-0.6599	-0.1669	1.2509	-1.5183
0.5500	-0.1594	0.9830	-0.6584	-0.1680	1.2503	-1.5178
0.5600	-0.1604	0.9823	-0.6570	-0.1690	1.2496	-1.5174

Figure 27: Joint angular position (radians)

1.4. MATLAB simulation

A Simscape Multibody model of the 6-DOF GOFA robot was created to visualize the trajectory tracking task. The model includes all six revolute joints; each connected to physical links and actuated using joint angle inputs derived from Task 1c. The end effector's position is monitored through output ports, which extract the X, Y, Z coordinates in a quantized discrete timestep format. The robot's movement was animated and exported to a video file, as required for Task 1d. The model also includes switches to test custom inputs for each joint.

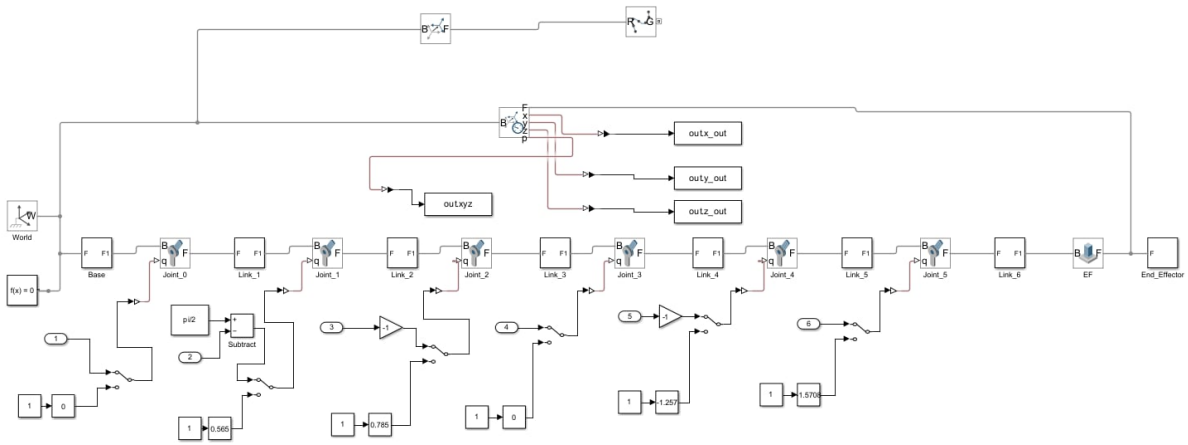


Figure 28: Simulink model of GOFA5

Model explanations:

Before inputting the joint angle data into the model, it is necessary to test whether the simulation correctly receives the input. This is done by consecutively testing the initial joint angles from joint 1 to joint 6 to verify that the robot assumes the correct initial pose.

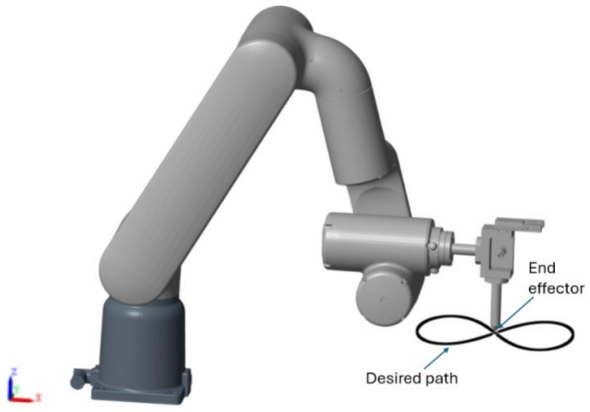


Figure 29: Expected initial pose of the robot

When all joints' angles are equals to 0 the robot has such pose, which perfectly matches the original pose in solid works file.

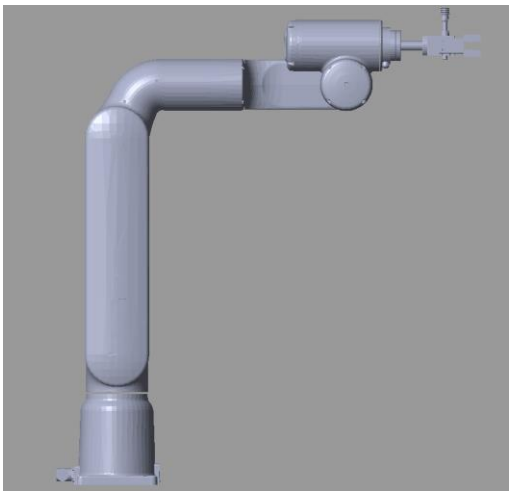


Figure 30: When all joints are set to 0 degree of rotation

For theta1 = 0 and theta2 = pi/3 the robot has such pose which does not match with the pose in fig.

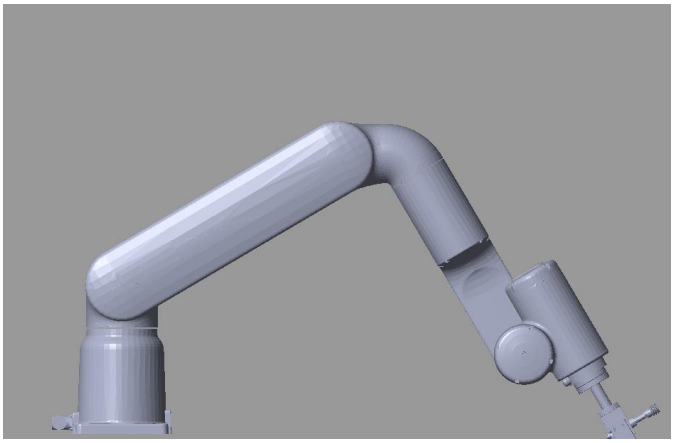


Figure 31: When joint 2 is set to initial angle (pi/3) and joint 3 is 0

To address this problem, we change theta2 = 0 to see the different between the orientation of second joint and the joint in fig. 29.

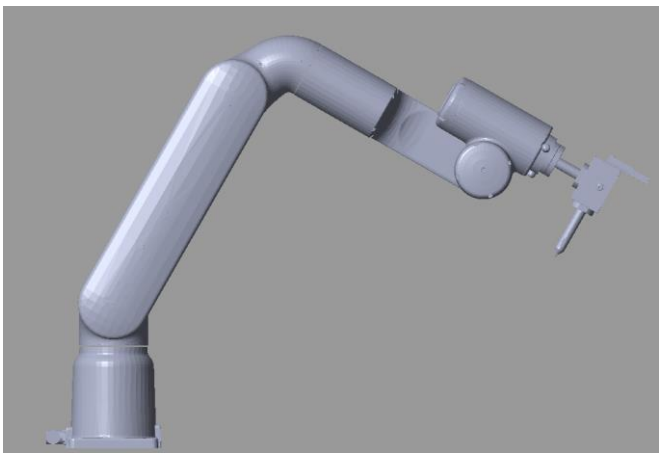


Figure 32: When joint 2 is set to 0

The Z-axis orientation of the second joint appears to be reversed compared to the corresponding joint in the example robot, and the X-axis is offset by 90 degrees. This discrepancy is illustrated in the figure below, which compares the behaviour of the two joints.

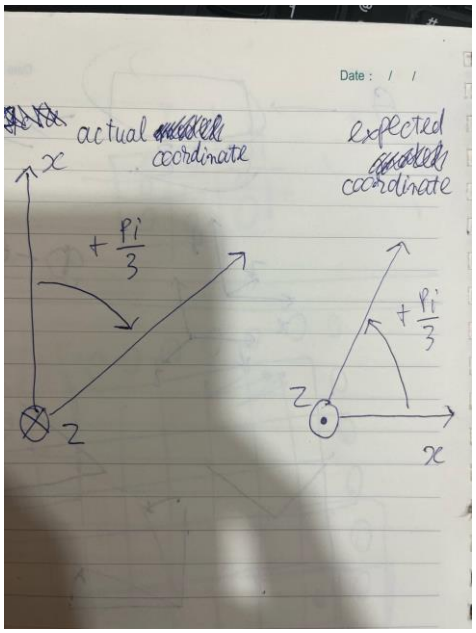


Figure 33: Expected coordinate and actual coordinate of second joint

To correct this mismatch, a gain block is added to compute π/2−theta2 , ensuring alignment with the expected behaviour.
A similar issue is observed with the third joint. When theta3 = −π/4 is applied, the robot's pose deviates from that shown in the example figure.

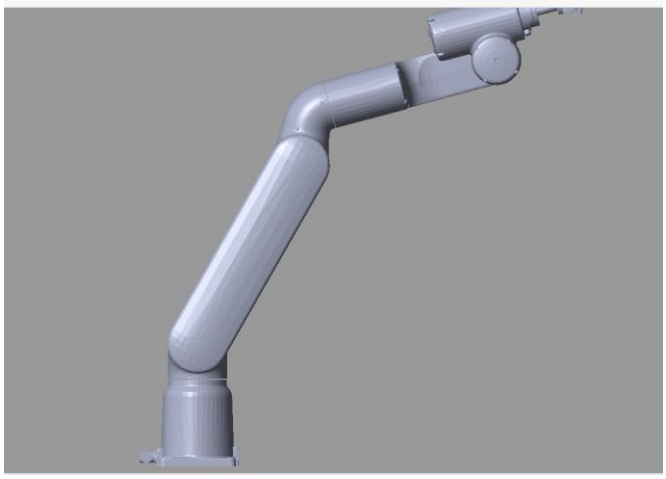


Figure 34: When joint 3 is set to initial angle (-pi/4)

To analyze this, theta3 is temporarily set to 0 (figure 32), allowing for a clearer comparison of the third joint's orientation. The analysis reveals that the Z-axis of this joint is also reversed compared to the reference model.

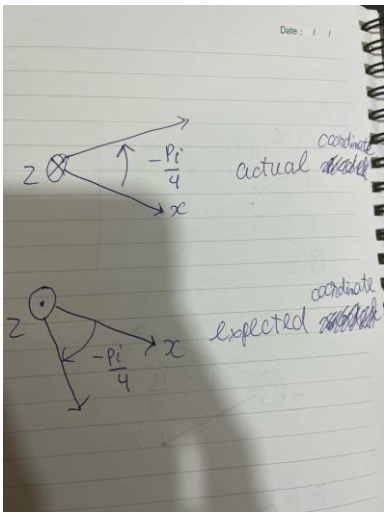


Figure 35: Actual and expected coordinate of joint 3

To address this, a gain block is added to invert the angle, using theta_3 = -theta3.
The same pattern occurs with the fifth joint. When theta5 = 5π/12 is applied, the resulting pose again differs from the example configuration.

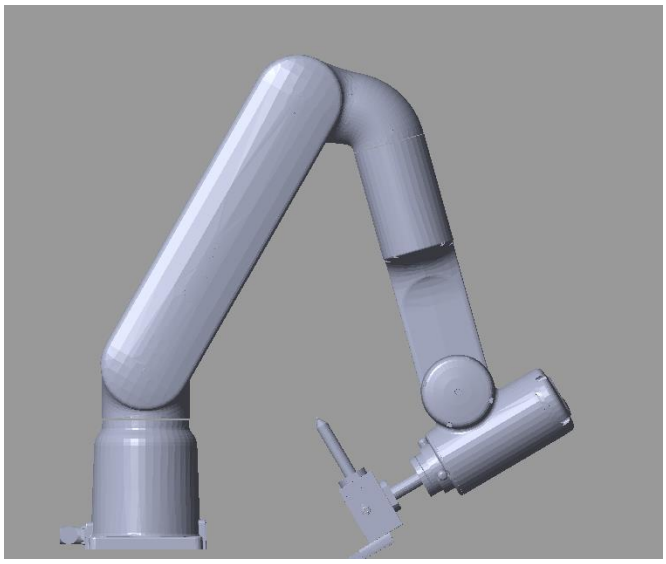


Figure 36: When joint 5 is set to initial angle (5pi/12)

Setting theta_5 = 0 helps reveal the misalignment, showing that the Z-axis is reversed in this case as well.

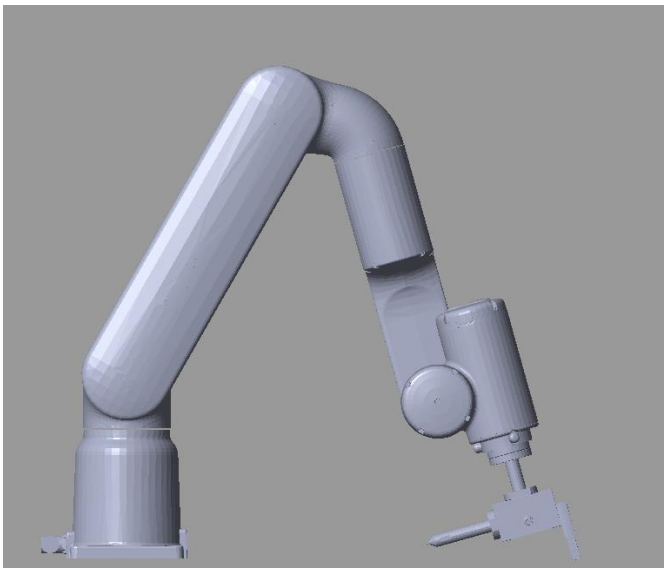


Figure 37: When joint 5 is set to 0 degree

A gain block is thus introduced to compute theta_5 = -theta_5 for proper alignment.

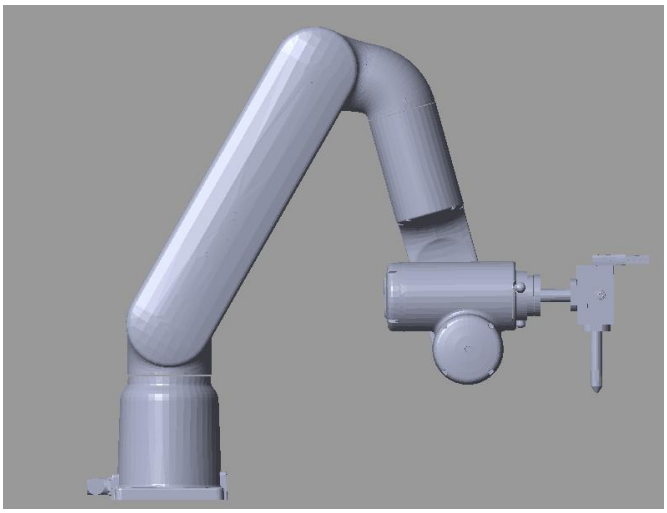


Figure 38: Final pose

For the remaining joints, the robot aligns correctly with the initial configuration and no further adjustments are required.

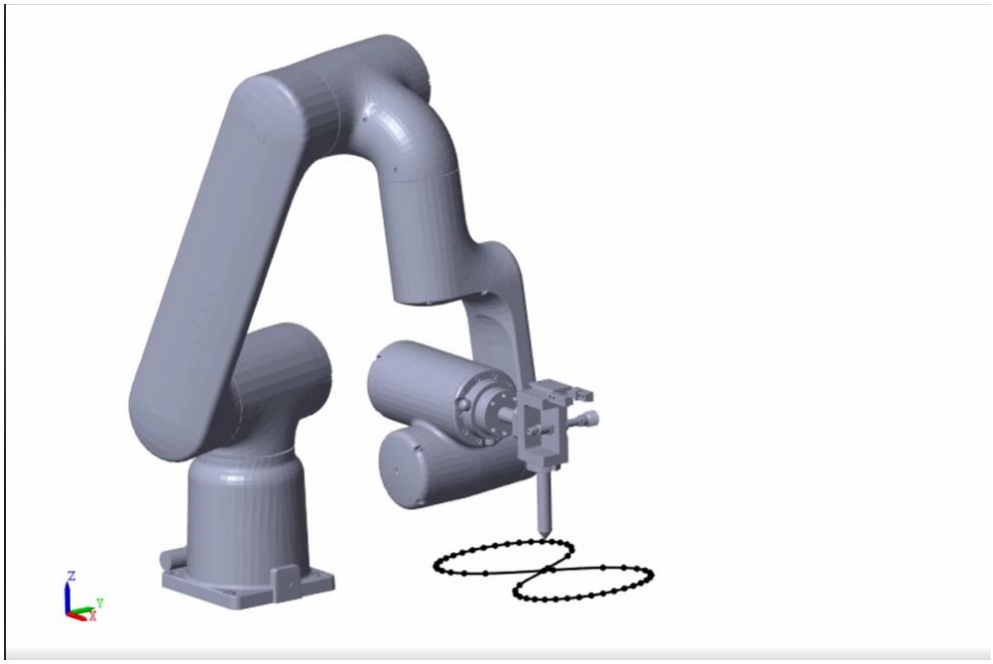


Figure 39: Simulation of GOFA5 writing 8-shape figure

Future work:

While investigating the cause of the robot's unexpected behaviour, an important observation emerged. When all joint angles are set to zero, the resulting robot pose (figure) perfectly matches the original pose defined in the SolidWorks model. However, the expected configuration—based on the DH parameter table—requires joint 2 to be rotated by 90 degrees when all other joints are at zero. Since the DH parameters were verified in Section 1 of Problem 1, the code implementation is assumed to be correct. This discrepancy is likely caused by an internal error or a default setting in Simscape that alters the joint orientations. As a potential solution, future work should involve generating custom trajectory data tailored to the robot's actual configuration.

Question 2

2.1. DEMO TASK: Palletising

2.1.1. Palletising Task Summary – RMIT Education Cell

The **palletising** function is one of the key automated operations in the Gofa Cobot Education Cell. It simulates an industrial use case where a robot picks products from a **slider** and places them on a **pallet** to be further processed in the automation line.

Functional Overview

- **Purpose:** Automatically pick objects (white acrylic rectangular blocks) one by one from the slider and stack on the pallet.
- **Placement Logic:** Items are organized into either **even** or **odd** positions on the pallet.

2.1.2. System Hardware Components

RMIT Cobot Education Cell

The RMIT Cobot Education Cell is a variation of the GOFA Robot Education Cell which is a training platform designed to create a safe hands-on learning experience that is engaging for students looking to gain experience in the field of automation. This collaborative robot system features a user-friendly interface that allows users to easily program and control the robotic arm for various task such as handling and manipulating objects. In this instance, the focus will be on understanding and reverse engineering the process of **palletising**.

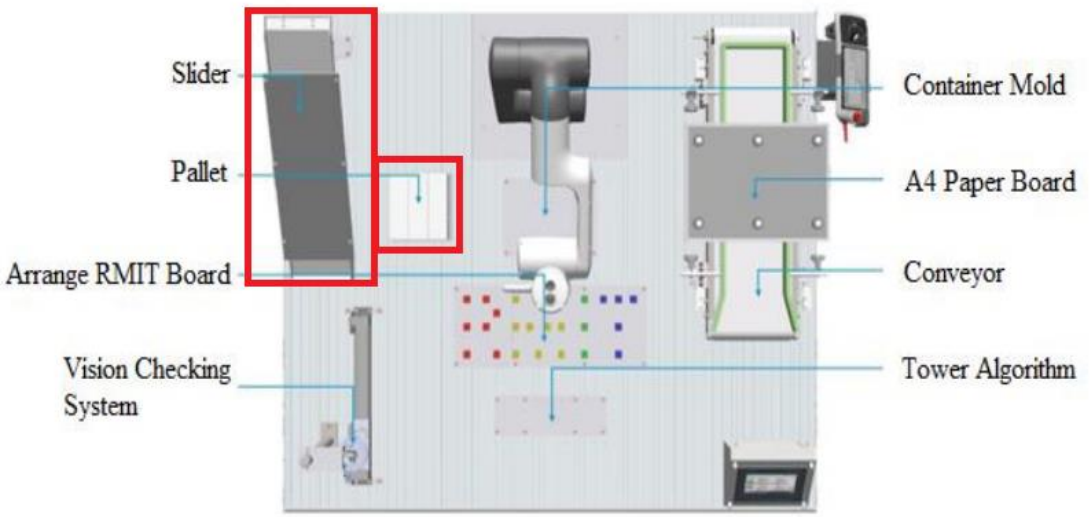


Figure 40: RMIT Cobot Education Cell Work Surface

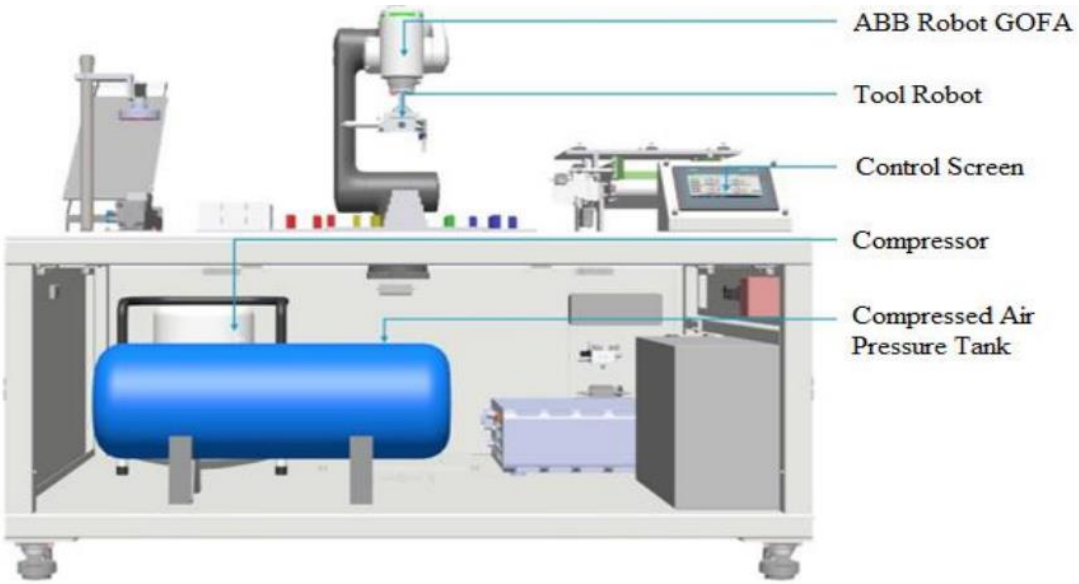


Figure 41: RMIT Cobot Education Cell Internal Layout

- **Function:** Encapsulates all the necessary subsystems (i.e. Robotic Arm, Slider, Air Compressor, etc) and provides a metal work surface for the pallet to be stacked.

- **Integration:** Ensures a fixed position for the robotic arm, slider, and pallet in an organized structure that can be easily mapped in the x,y,z coordinate frame.

ABB GOFA Robot (CRB 15000)

- **Function:** The primary function of the GOFA robot in the palletising task is to automate the picking and placement of items from the slider to the pallet. It acts as the main manipulator, using its robotic arm to **grasp and move** objects from the bottom of the slider to an arrangement to the designated location (pallet).

- **Integration:** It **receives object coordinates and orientation data** from the controller. A motion planning algorithm then computes safe, collision-free paths for the GOFA's arm. The robot's end-effector (vacuum gripper) is tailored for suction of the rectangular blocks. The system uses a PLC (programmable logic controller) to synchronize movements, monitor status, and interface with other parts of the automation system.

HMI (Human Machine Interface)



Figure 42: FlexPendant (Left) and Main Control Panel (Right)

- **Function:** The FlexPendant is used to **manually control and teach** the robot. It allows the user to **jog the robot**, set waypoints, and monitor real-time operations. This provides access to **programming and safety settings** enabling easy setup of the palletising task and adjustments. The Main Control Screen displays a **visual overview of the system**, including the status, sensor feedback, and process flow. This enables **monitoring and troubleshooting**, helping users understand the robot's behaviour and performance during palletising. Both the FlexPendant and Main Control Screen include buttons to **start, pause, or stop** the palletising task and allow switching to other modes and task.

- **Integration:** Together, the HMI and Control Screen are integrated to streamline robot setup and execution of the palletising task. The user can define the palletising routine (e.g. pickup points, drop-off zones) via the HMI.

Pneumatic System

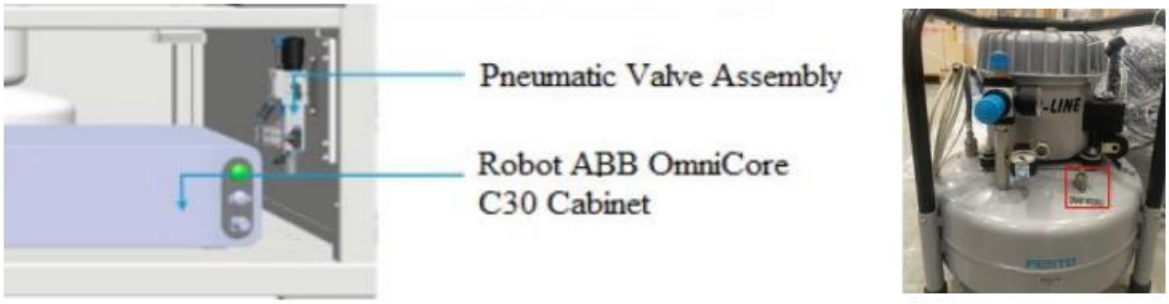


Figure 43: Pneumatic Valve (Left) and Air Compressor with Tank (Right)

- **Function:** The vacuum head is an end effector mounted on the robot arm, it creates suction to **grip and lift** items from the pallet surface without requiring mechanical fingers or claws. The air compressor generates the compressed air to power the vacuum system. The compressed air pressure tank stores pressurized air to ensure a stable and consistent air supply for reliable vacuum performance. The solenoid valve **electronically controls** the flow of compressed air to the vacuum head, allowing the robot to **engage or release suction** on command.

- **Integration:** The pneumatic system is **controlled in coordination with the robot's motion sequence**. When the robot moves to pick up an item, the **controller triggers**

the solenoid valve, activating suction at the vacuum head through the means of the venturi effect. Once the rectangular block is securely held, the robot moves it to the drop off point (top of the slider). The solenoid valve is then switched off to **release the object**.

Slider

- **Function:** The slider acts as a gravity fed linear guided track that serves as the **source location** for the rectangular stacked items that the robot will deliver to pallet during the palletising task.

- **Integration:** The bottom of the slider position is hard coded into the palletising task and is **accessible** by the reach of the robot. This enables the robot to quickly pick-up the rectangular stacked items one by one and place them on the pallet.

Pallet

- **Function:** The pallet **holds** the stacked rectangular items that the robot will stack up during the palletising routine.

- **Integration:** The pallet serves as the **destination location** for the robot's pick-and-place operation. The pallet's position is stored in the palletising routine allowing the robot to effectively move to the correct location to place the stacked rectangular components.

Rectangular Stacked Objects

- **Function:** These items are the **simulated objects** that might otherwise be manipulated or interacted with in an industrial environment.

- **Integration:** In the palletising task, these items are picked up one by one from the slide, then moved to be stacked on the pallet in an even and odd fashion of 4 layers with each layer containing 3 items side by side.

2.1.3 Palletising Procedure RAPID Code Analysis

Introduction

The palletising task is a routine that is coded in RAPID, a high-level programming language used to control an ABB collaborative robot in the RMIT Educational Cell. It automates the task of palletising rectangular blocks by using a vacuum gripper, sensor inputs, and a series of movement sequences with the robotic arm. This procedure is designed to pick 12 items from the slider and place them onto a pallet in a 2-layer, 3 x 2 pattern (3 items in one orientation, then 3 items rotated 90 degrees, per layer).

Palletizing RAPID Code Overview

The overall RAPID code necessary to execute the palletising task is organized into **three main modules**. While these modules contain many other variables and other procedures,

the main focus will be on the code within these modules that are related to the procedure of palletising.

1. **MODULE Wizard:** This is the main application module.

PROC main(): The entry point. Calls initialization, homing (routine rHome) and the main application loop.

PROC rLoop():

- Sends the robot home and clears the Teach Pendant screen.
- Signals a "Job Request" to an external system (PLC) and waits for a "Start Trigger" input (diStart_Trigger).
- Reads a model number (giModel_Nr) to determine which "lesson" or sub-program to run.
- Uses an IF/ELSEIF structure to call different procedures based on nModel:
    1. rP1_Drawing (Model 3)
    2. **rP2_Palletizing_And_Depaletizing (Model 1) – This is our focus.**
    3. rP3_Color_Classification (Model 2)
    4. rP4_Stack_Tower (Model 4)

PROC rP2_Palletizing_And_Depaletizing():

- Handles the core **logic for executing both palletising and depalletizing**.
- Performs an initial handshake with an external system (doRB_Job_Complete, diJob_Valid, doRB_Job_Request).
- Moves the robot to a general wait position pP2_wait.
- **Palletising Branch** (IF diRB_P1_Palletizing=1):
    1. If Palletizing_done is TRUE, it signals an error/completion and loops.
    2. Otherwise, it moves to pP2_Wait_slide, calls slideToPallet to perform the palletising, returns to pP2_Wait, signals job completion, and loops.

PROC slideToPallet(): This is the **core palletising routine**, which is described in detail further below.

- Manages count_sileToPallet to track progress.
- Divided into four IF blocks, each handling 3 items for a total of 12.
- Picks items from pickSlide.
- Places items onto a pallet using putPallet and putPallet_2 (for 90-degree rotated items).
- Handles different Z heights for two layers.
- Includes vacuum control and verification
- Sets Palletizing_done when all 12 items are stacked.

2. **MODULE M2_Teaching_PositionRecipe:** This module primarily stores taught robot positions (robtarget) as persistent variables.

Numerous PERS robtarget declarations for various wait positions, pick/place points for different "lessons" (P1, P2, P3, P4).

Key points are for P2: pP2_Wait, pP2_Wait_pallet, pP2_Wait_slide, putPallet, pickSlide

PERS wobjdata: wobj_pallet (work object definitions).

PROC rPositionRecipe_1():

- Defines CONST robot targets (local and recipe-specific points).
- Crucially, it defines putPallet_2 based on putPallet with an offset and a -90 degree rotation around Z:

  putPallet_2:=RelTool(putPallet,-53.37,64.073,-20\Rx:=0\Ry:=0\Rz:=-90);

  The X offset here (-53.37) is slightly different from the one used in slideToPallet (-54.87) for the RelTool definition of putPallet_2.

3. **MODULE M0_Function_General:** Contains general utility functions

PROC rInitialize(): Calls iVersion, rSetting, and then CallByVar "rPositionRecipe_", 1;. This dynamically calls rPositionRecipe_1() from the M2 module to load the default position set.

PROC rSetting(): Initializes various digital outputs to OFF, group outputs to 0, and resets counters (count_sileToPallet) and boolean flags (Palletizing_done) to their default states. This is crucial for a clean start.

PROC rRESET_DO(): Resets a subset of digital outputs and variables, similar to rSetting.

PROC rTool(string sToolAct): A very important procedure to control tools on the end effector.

- Handles "VACUUM_ON" and "VACUUM_OFF"

Detailed Procedure Code Analysis (slideToPallet)

Execution (12 items, 2 layers, 3 x 2 per layer with rotation):

Key Variables & Parameters:

- **count_sileToPallet:** Counter that tracks items palletised (0-11). Initialized to 0 in rSetting.
- **palletizing_done:** Flag that is set to TRUE when 12 items have been stacked
- **pickSlide:** Robot target for picking item from the slide.
- **putPallet:** Base robot target for placing items (first layer, 0-degree orientation).
- **putPallet_2:** Derived robot target for placing items rotated -90 degrees.
- **value_offset:** A Z-offset for approach and retract moves
- **oSS_VAC:** Digital input for vacuum sensor

Initialization:

- Checks if the variable count_sileToPallet is equal to zero, then sets palletizing_done to FALSE.

Items 1-3 (count_sileToPallet = 0, 1, 2), First Layer, 0-degree

- Uses a FOR loop to pick up the first 3 rectangular objects from the bottom of the slider
- The robot approaches the bottom of the slider, turns on the vacuum, and verifies the oSS_VAC state
- Then the robot goes to place the rectangular object it just picked up by moving to the pP2_Wait_pallet position.
- The place X-offset: (-60.5*i)–10 (approach X), then (-60.5*i) (final X) from putPallet.
- Vacuum OFF, verify the state of oSS_VAC, then retract.
- Increments count_sileToPallet before continuing the next for loop iteration.

Items 4-6 (count_sileToPallet = 3, 4, 5), First Layer, -90-degree

- Again, the code uses a FOR loop to pick up the next 3 rectangular objects from the bottom of the slider
- The robot follows the same pick logic to approach the bottom of the slider, turns on the vacuum, and verifies the oSS_VAC state
- Similarly to the items 1-3, the robot moves to pP2_Wait_pallet position.
- This time the code utilizes a Move Linear command that uses putPallet_2 as the base.
- There is a Y-offset: (60.5*j)+10 (approach Y), then (60.5*j) (final Y) from putPallet_2
- Alongside the rectangular objects being rotated negative 90 degrees, there is a Z-height determined by the variable value_offset
- Once again the vacuum is turned off, the vacuum sensor is verified, and then the robot retracts.
- count_sileToPallet is incremented in the loop before continuing the next iteration.

At this stage of the program, the subsequent two FOR loops are functionally equivalent to the initial two loops responsible for stacking the first layer of rectangular objects on to the pallet. The primary distinction in the logic governing the stacking of the second layer (Items 7-9 and Items 10-12) lies in the Z-height offset values, which are adjusted to account for the increased height resulting from the completion of the first layer.

Upon completion of the fourth FOR loop — signifying the successful placement of the second layer — the variable count_sileToPallet reaches a value of 12. This triggers a conditional branch in the program that sets the Boolean variable Palletizing_done to TRUE and outputs the message “Palletizing Finish” to the touch pendant console.

The slidetoPallet procedure concludes by executing a Move Joint instruction, directing the manipulator to the predefined pP2_Wait_pallet position.

```
PROC slideToPallet()
    IF count_sileToPallet=0 THEN
        palletizing_done:=FALSE;
        count_sileToPallet:=0;
    ENDIF
    MoveJ pP2_Wait_slide,vWait,fine,tVacuum\WObj:=wobj0;
    nCurrent_Position:=201;
    IF count_sileToPallet<3 THEN
        MoveJ pP2_Wait_slide,vWait,fine,tVacuum\WObj:=wobj0;
        nCurrent_Position:=201;
        FOR i FROM count_sileToPallet TO 2 DO
            MoveJ pP2_Wait_slide,vWait,z100,tVacuum\WObj:=wobj0;
            nCurrent_Position:=201;
            rpickSlide:
            MoveL RelTool(pickSlide,0,0,-value_offset),vWait,z100,tVacuum\WObj:=wobj0;
            nCurrent_Position:=210;
            MoveL pickSlide,vWork,fine,tVacuum\WObj:=wobj0;
            nCurrent_Position:=211;
            rTool("VACUUM_ON");
            WaitTime 1;
            MoveL RelTool(pickSlide,0,0,-value_offset),v20,z10,tVacuum\WObj:=wobj0;
            nCurrent_Position:=210;
            IF oSS_VAC=0 THEN
                GOTO rpickSlide;
            ENDIF
            MoveJ pP2_Wait_slide,vWait,z10,tVacuum\WObj:=wobj0;
            nCurrent_Position:=201;
            MoveJ pP2_Wait_pallet,vWait,z0,tVacuum;
            nCurrent_Position:=201;
            MoveL Offs(putPallet,(-60.5*i)-10,0,value_offset),vWait,z10,tVacuum\WObj:=wobj0;
            nCurrent_Position:=220;
            MoveL Offs(putPallet,(-60.5*i)-10,0,15),vWait,z10,tVacuum\WObj:=wobj0;
            nCurrent_Position:=221;
            MoveL Offs(putPallet,-60.5*i,0,15),v100,fine,tVacuum\WObj:=wobj0;
            nCurrent_Position:=221;
            MoveL Offs(putPallet,-60.5*i,0,0),vWork,fine,tVacuum\WObj:=wobj0;
            nCurrent_Position:=221;
            rTool("VACUUM_OFF");
            WaitTime 1;
            IF oSS_VAC=1 THEN
                SetGO Go_Error,8;
                TPErase;
                TPWrite "Tool Vacuum OFF Error";
                Stop\AllMoveTasks;
            ENDIF
            count_sileToPallet:=count_sileToPallet+1;
            TPWrite "Pick_slideToPallet:"\num:=count_sileToPallet;
            MoveL Offs(putPallet,-60.5*i,0,value_offset),vWait,z10,tVacuum\WObj:=wobj0;
            nCurrent_Position:=220;
            MoveJ pP2_Wait_pallet,vWait,z10,tVacuum;
            nCurrent_Position:=201;
        ENDFOR
    ENDIF
```

Figure 44: slideToPallet Procedure First FOR Loop (Items 1-3)

Palletising Logic Flowchart

Overall, the logic of the palletising procedure is relatively straightforward. There are a lot of repetitive steps with minor differences in the iterations of the FOR loops, namely the offsets for stacking the pallet in the Z-axis, and rotation of the objects in the stack depending on the current layer and object count. The flowchart below encapsulates the slideToPallet procedure. Some parts of the routine are such as the vacuum grippers state and whether it was successful in picking up the object was merged to save some space in the flowchart. The output of that IF block is further then divided again based on the current count of the objects.

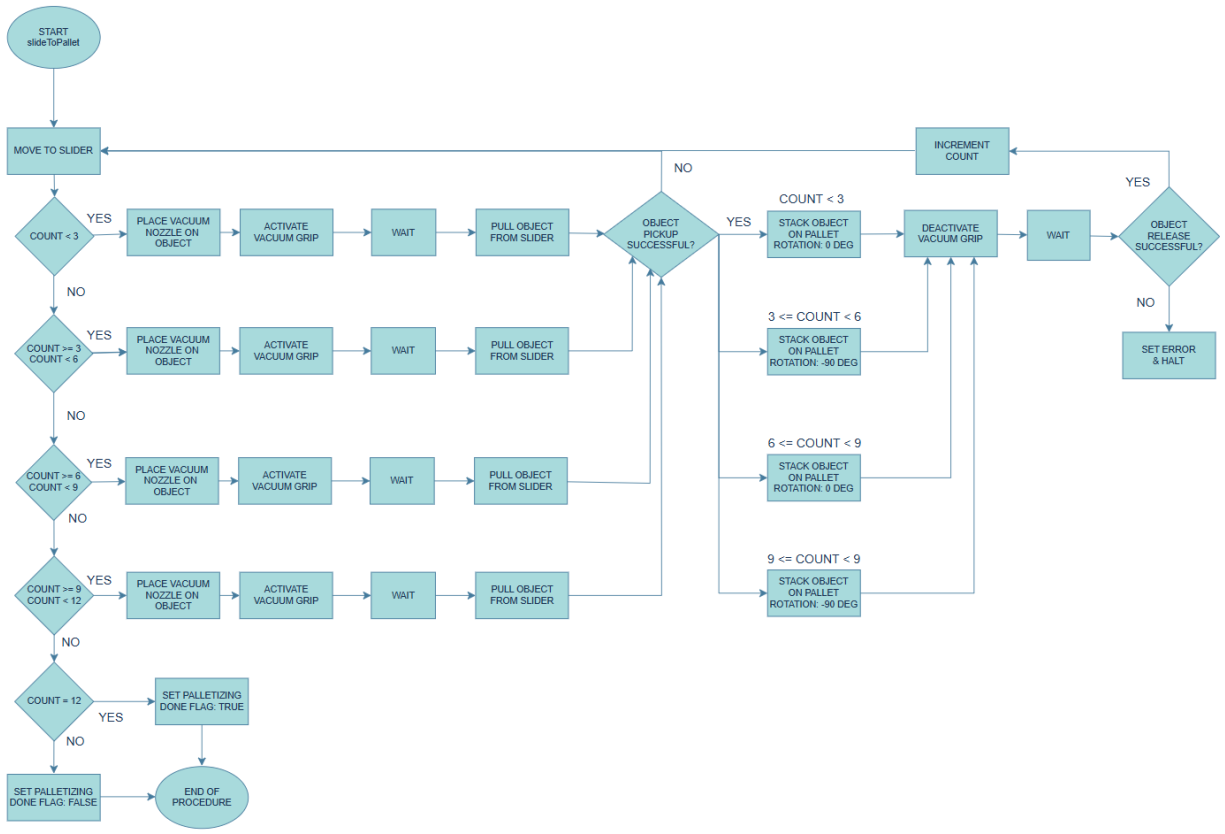


Figure 45: Palletising Subroutine Flowchart

Summary of Logic

The program initializes the robot, then enters a loop where it waits for user selection. Depending on the selected model number, it runs the corresponding task. For palletising, it picks objects from the bottom of the slide and places them on a pallet in layers. The logic ensures error checking (e.g. vacuum status) and provides clear operator prompts via the touch pendant. If the vacuum head fails to pick up the object, the program jumps back to an earlier instruction to reattempt to pick up the object from the slider. If the vacuum head fails to release the object onto the stacked pallet, then the program sets an error, notifies the user through the touch pendant, and stops all movement of the robot. Until all 12 items have been stacked on the pallet, the program does not set the palletising done flag to true.

2.1.4 Palletising RAPID Code Performance Evaluation

Pros:

- Comprehensive Tool Control: rTool is a robust procedure for handling the vacuum gripper and includes checking and retrying for the gripper.
- The use of PERS for Position: Storing robtargets as PERS makes them retain their values across power cycles.
- Centralized Initialization: rSetting in the M0_Function_General provides a clean way to initialize system state, counters, and flags.
- Dynamic Position Loading: CallByVar “rPositionRecipe” allows for different sets of positions (“recipes”) to be loaded.
- Resumability: The use of count_sileToPallet in the FOR loop initialization allows the possibility for the process to be resumed if interrupted.
- Clear Status Updates: The use of TPWrite messages are used to indicate progress and errors.

Cons:

- High Code Repetition: The four IF blocks for palletising are nearly identical. This is the most significant drawback, making the code long, hard to read, and very difficult to maintain. A change in pick/place logic needs to be replicated four times.
- Initialization of Counters: There is a redundant reset of the variable count_sileToPallet at the beginning of slideToPallet which is unnecessary because it is already reset in rSetting.
- Error Messages: The message “Palletizing done please change depalletizing” appears in both palletising and depalletizing completion blocks when instead the depalletizing one should say “Depalletizing done…”.
- Redundant Moves: Some consecutive MoveJ to the same point (e.g. MoveJ pP2_Wait_slide twice at the start of the first IF block in slideToPallet).

Improvement Suggestions:

- Refactor slideToPallet. Use a single FOR loop from 0 to 11. Inside the loop, calculate the layer, row, column, orientation, and target Z-heights based on the loop index(i).
- Create a helper procedure for PickFromSlide and PlaceOnPalletAtCalculatedPosition.
- Remove the redundant reset. Only palletizing_done:=FALSE; is needed if count_sileToPallet is 0.
- Fix the name of count_sileToPallet, to what is most likely intended to be count_slideToPallet.
- Review and remove unnecessary moves.

2.2. GOFA Robot Writing Operation

2.2.1 Objective Overview and Planning Process

This exercise involves developing a RAPID program to control the ABB GOFA robotic arm in writing the first initials of the team members’ names “M” for Minh and “N” for Nguyen

using a pen tool as the end-effector. The task requires the robot to accurately trace the shapes of the letters “M” and “N” on an A4 sheet, utilizing a defined Cartesian coordinate system (X, Y, Z). To enable precise execution, each letter was decomposed into a series of linear segments, as both characters consist of straight lines. Each segment is specified by distinct start and end coordinates in the X-Y plane, measured relative to the bottom-left corner of the A4 paper, which serves as the origin. This approach ensures structured motion planning and consistent drawing behaviour by the robotic arm.

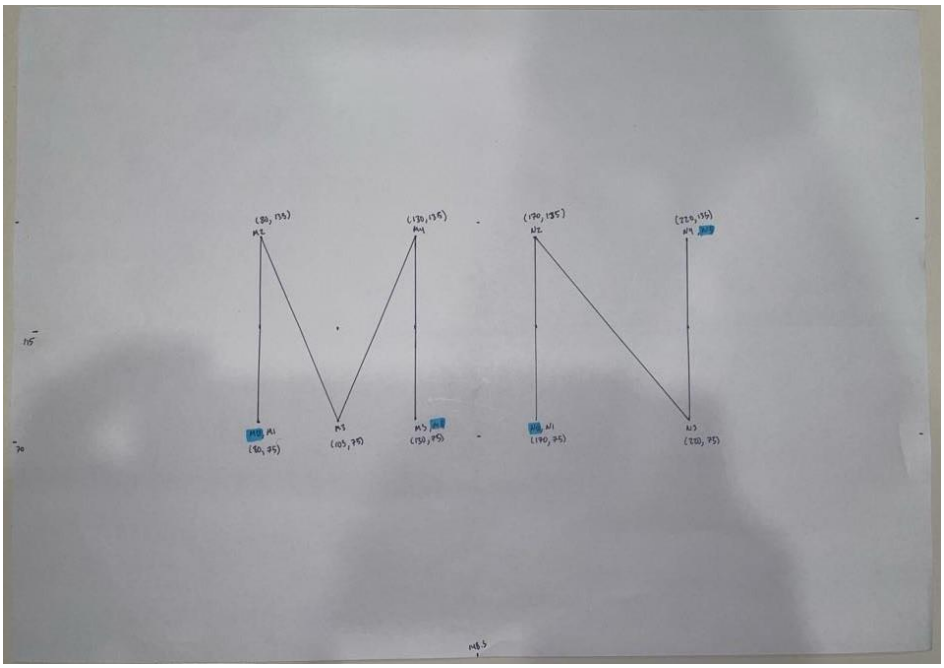


Figure 46: Letters MN Target Point Planning

In the figure above, the target coordinates of each point of the linear segments to draw the letters “M” and “N” were measured in millimetres from the bottom left corner of the paper. These X and Y coordinates were then utilized in writing the RAPID code to define the target positions which the robot arm would follow. The points highlighted in blue contained target positions where the robot would either move to with the pen tip above the paper or lift the pen tip away from the paper, while the remaining points denoted where the pen tip would touch the paper to create the strokes.

2.2.2 RAPID Code Development

Defining Position and Target Points

The RAPID code defines multiple persistent robtarget points that guide the ABB GOFA robot arm during the drawing task. Each robtarget represents a specific position and orientation of the robot’s tool in 3D space. The Cartesian coordinate system defines the starting and ending point of each letter on the A4 sheet, with the bottom-left corner of the page set as the origin. The orientation of the end effector is managed using quaternion values. The Z-coordinate determines the pen’s height above the paper surface, where a negative value allows for the pen tip to be pressed down onto the paper, allowing for complete stroke formation. Configuration settings define the robot’s joint positions for

each target, while external axes are kept at default values. These programmed parameters enable the robot to perform the motions required for drawing the letters.

```
PERS robtarget M0:=[[25.0,50.0,100.0],[0.0197915,0.998036,0.0594276,-0.000234469],[0,1,-3,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];
PERS robtarget M1:=[[25.0,50.0,-3.48],[0.0197915,0.998036,0.0594276,-0.000234469],[0,1,-3,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];
PERS robtarget M2:=[[25.0,200.0,-3.48],[0.0197915,0.998036,0.0594276,-0.000234469],[0,1,-3,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];
PERS robtarget M3:=[[75.0,50.0,-3.48],[0.0197915,0.998036,0.0594276,-0.000234469],[0,1,-3,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];
PERS robtarget M4:=[[125.0,200.0,-3.48],[0.0197915,0.998036,0.0594276,-0.000234469],[0,1,-3,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];
PERS robtarget M5:=[[125.0,50.0,-3.48],[0.0197915,0.998036,0.0594276,-0.000234469],[0,1,-3,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];
PERS robtarget M6:=[[125.0,50.0,20.0],[0.0197915,0.998036,0.0594276,-0.000234469],[0,1,-3,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];

PERS robtarget N0:=[[180.0,50.0,20.0],[0.0197915,0.998036,0.0594276,-0.000234469],[0,1,-3,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];
PERS robtarget N1:=[[180.0,50.0,-3.48],[0.0197915,0.998036,0.0594276,-0.000234469],[0,1,-3,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];
PERS robtarget N2:=[[180.0,200.0,-3.48],[0.0197915,0.998036,0.0594276,-0.000234469],[0,1,-3,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];
PERS robtarget N3:=[[260.0,50.0,-3.48],[0.0197915,0.998036,0.0594276,-0.000234469],[0,1,-3,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];
PERS robtarget N4:=[[260.0,200.0,-3.48],[0.0197915,0.998036,0.0594276,-0.000234469],[0,1,-3,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];
PERS robtarget N5:=[[260.0,200.0,10.0],[0.0197915,0.998036,0.0594276,-0.000234469],[0,1,-3,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];
```

Figure 47: Letters MN Persistent Target Definitions

To facilitate structed programming, each persistent target was systematically labelled according to the corresponding letter being drawn, with indices incrementing sequentially. As illustrated in the figure above, the robot initiates motion to target M0, located relative to the origin point (bottom-left corner of the A4 paper), where the pen tip is positioned 100 millimetres above the paper surface. The subsequent target, M1, maintains the same X (25.0 mm) and Y (50.0 mm) coordinates as M0; however, the Z-axis value is set to -3.48 mm to ensure the pen tip contacts the paper, enabling ink deposition.

The drawing sequence proceeds through targets M1 to M5, with the robot executing linear movements while maintaining contact between the pen tip and the paper to form the strokes of the letter “M”. At M6, the X and Y coordinates are consistent with those of M5, but the Z value is raised to 20.0 mm, lifting the pen to transition to the starting point of the next character, “N”. The drawing of the letter “N” follows a comparable trajectory-based approach, though it requires fewer target points due to its simpler geometry.

Defining the Procedure

The trajectory planning for this task is implemented within the *word_write* procedure of the *RMIT_HOME* module using the RAPID programming language. Given the geometry simplicity of the letters “M” and “N”, which consist solely of linear segments, the implementation primarily utilizes two standard robot motion instructions: **MoveJ (Joint Movement)** and **MoveL (Linear Movement)**. These commands dictate the manner in which the robot transitions between defined target positions.

- **MoveJ (Move Joint):**

  This instruction moves the robot to a specified target position by interpolating through joint space. Each joint adjusts its angle independently to reach the target configuration in an efficient manner. As the resulting trajectory of the end-effector is not linear, it is governed by the robot’s kinematics rather than a straight-line path in Cartesian space. With in this application, MoveJ is primarily employed for initial positioning of the pen tool above the drawing surface.

- **MoveL (Move Linear):**

This command generates a straight-line trajectory of the end effector from its current position to the target coordinate in Cartesian space. It ensures the tool follows a precisely defined path, which is critical for task involving continuous contact with a surface, such as writing or engraving. The *word_write* procedure makes extensive use of the MoveL to enable accurate tracing of each letter stroke on the A4 sheet.

Additionally, parameters are used to refine motion control and sure task precision:

- 'V100' and 'Vdraw' specify movement speed. V100 represents a standard velocity, while Vdraw denotes a slower, more precise speed used during the actual drawing phase to maintain trajectory accuracy.

- 'fine' define the tool data, representing the pen tool mounted on the robot. It sets the tool center point (TCP) for motion calculations.

- '\WObj:=wobj_penVBOT' assigns a specific workobject, which establishes the reference coordinate system for motion commands. In this case, the workobject corresponds to the A4 paper sheet, position its bottom-left corner as the origin [0,0,0]. This allows all motion targets to be interpreted relative to the paper rather than the robot's global base frame.

Together, these motion instructions and parameters settings enable precise and structured robot movement, ensuring that the end-effector accurately reproduces the desired letter forms on the paper surface.

```
PROC MN_word_write()
    MoveJ p_ready_to_write,vWait,fine,tPen\WObj:=wobj_penVBOT;
    ! Move to the upper position
    MoveL M0,v100,fine,tPen\WObj:=wobj_penVBOT;
    ! Start writing
    MoveL M1,v30,fine,tPen\WObj:=wobj_penVBOT;
    MoveL M2,Vdraw,fine,tPen\WObj:=wobj_penVBOT;
    MoveL M3,Vdraw,fine,tPen\WObj:=wobj_penVBOT;
    MoveL M4,Vdraw,fine,tPen\WObj:=wobj_penVBOT;
    MoveL M5,Vdraw,fine,tPen\WObj:=wobj_penVBOT;
    MoveL M6,Vdraw,fine,tPen\WObj:=wobj_penVBOT;

    MoveJ N0,Vdraw,fine,tPen\WObj:=wobj_penVBOT;
    MoveL N1,V30,fine,tPen\WObj:=wobj_penVBOT;
    MoveL N2,Vdraw,fine,tPen\WObj:=wobj_penVBOT;
    MoveL N3,Vdraw,fine,tPen\WObj:=wobj_penVBOT;
    MoveL N4,Vdraw,fine,tPen\WObj:=wobj_penVBOT;
    MoveL N5,Vdraw,fine,tPen\WObj:=wobj_penVBOT;

    MoveJ p_ready_to_write,vWait,fine,tPen\WObj:=wobj_penVBOT;
ENDPROC
```

Figure 48: Letters MN Drawing Procedure Definition

MN_word_write() RAPID Code Logic Flow

1. Position the end-effector pen at the origin of the paper (bottom-left corner) with the orientation of the pen tip facing the paper.
2. Move the pen linearly to the M0 target with v100 velocity 100 mm above the paper.
3. Move the pen tip downward linearly to the M1 target to -3.48 mm in the Z-axis.
4. Draw a straight line to target M2 55 mm in the positive Y-axis from the point M1.
5. Draw a straight line from M2 to M3.
6. Draw a straight line from M3 to M4.
7. Draw a straight line from M4 to M5.
8. Lift the pen tip up to 20.0 mm above the paper at M6
9. Move the pen tip to target N0
10. Lower the pen tip to -3.48 mm in the Z-axis
11. Draw a straight line from N1 to N2.
12. Draw a straight line from N2 to N3.
13. Draw a straight line from N3 to N4.
14. Lift the pen tip 10.0 mm in the Z-axis
15. Move the pen back to the origin and wait there.

Command Components

The general syntax for defining a robtarget in RAPID for ABB robotic systems follows the structure:

```
PERS robtarget TargetName := [[X, Y, Z], [Q1, Q2, Q3, Q4], [CF1, CF4, CFX, CF6], [E1, E2, E3, E4, E5, E6]];
```

This construct encapsulates the spatial and configuration parameters required to accurately define a robot target. It consists of the four main components:

1. **Position**
   - The vector [X, Y, Z] specifies the Cartesian coordinates of the Tool Centre Point (TCP) in three-dimensional space.
2. **Orientation**
   - Defined by a quaternion [Q1, Q2, Q3, Q4], which represents the rotational component of the TCP.
   - For example, the quaternion [0.0197915, 0.998036, 0.0594276, -0.000234469] determines the precise orientation of the tool.
3. **Configuration Vector** [CF1, CF4, CFX, CF6]
   - **CF1:** Indicates the robot arm's base orientation or swing direction (e.g. left/right).
   - **CF4:** Specifies the wrist alignment (e.g. up/down positioning).
   - **CFX:** Refers to the flip state of the arm (e.g. overhand or underhand configuration).
   - **CF6:** Typically reserved as a default value for additional configuration (commonly set to 0).
   - A configuration vector such as [0, 1, -3, 0] implies:
     - Neutral arm base position (CF1 = 0),
     - Wrist oriented in a specific direction (CF4 = 1),
     - Arm flipped in a specific configuration (CFX = -3),
     - No additional configuration applied (CF6 = 0).
4. **External Axes**

- Represented by the array [9E+09, 9E+09, 9E+09, 9E+09, 9E+09, 9E+09], this field defines the positional values for up to six external axes, such as linear track of a rotary table.
- In the instance, the use of 9E+09 (an extremely large default value) indicates that no external axes are currently active or configured for this target.

This structured approach ensures precise spatial positioning and orientation of the robot's tool, as well as compatibility with multi-axis robotic systems.

2.2.3 Results and AI Validation

The RAPID code developed in this task was uploaded to the GOFA controller with a laptop connected over an ethernet connection. The procedure was set to the program pointer and then executed. The following figures display the AI verification and results of the letters drawn by the GOFA robot arm. The results shown below are from a subsequent run of the program with the Z-axis target positions set too high causing the last stroke of the letter "N" to be slightly incomplete as the pen tip did not make adequate contact with the surface of the paper.



Interpret what you see in the image

The image shows a white sheet of paper with the letters "M" and "N" written in thin, straight blue lines. The style of the letters is geometric, with each line segment being straight and minimalistic—almost like they were drawn with a ruler.

- The "M" consists of four straight lines forming a sharp and narrow "M" shape.
- The "N" is also composed of three straight lines, forming a clean and narrow "N".

There appear to be shadows on the paper, possibly from the person taking the photo, and the image might have been taken under indoor lighting. The letters likely represent the uppercase initials "MN." Let me know if you'd like a digital reconstruction or further interpretation.

Ask anything

ChatGPT can make mistakes. Check important info.

Figure 49: ChatGPT AI Verification

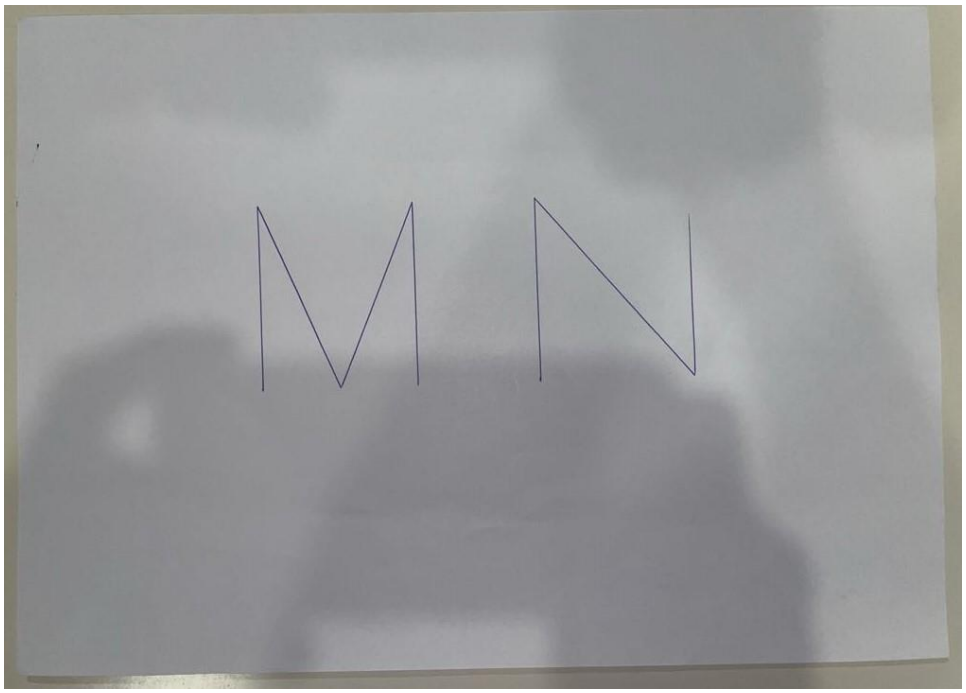


Figure 50: Letters MN written by the GOFA robot

Another set of RAPID code was developed and implemented on the GOFA robot to draw a set of letters with more complex geometry. Unfortunately, due to the time constraints and availability of access to the robot, the final attempt at executing that code was only semi-successful. Below is a figure of the path planning and results.

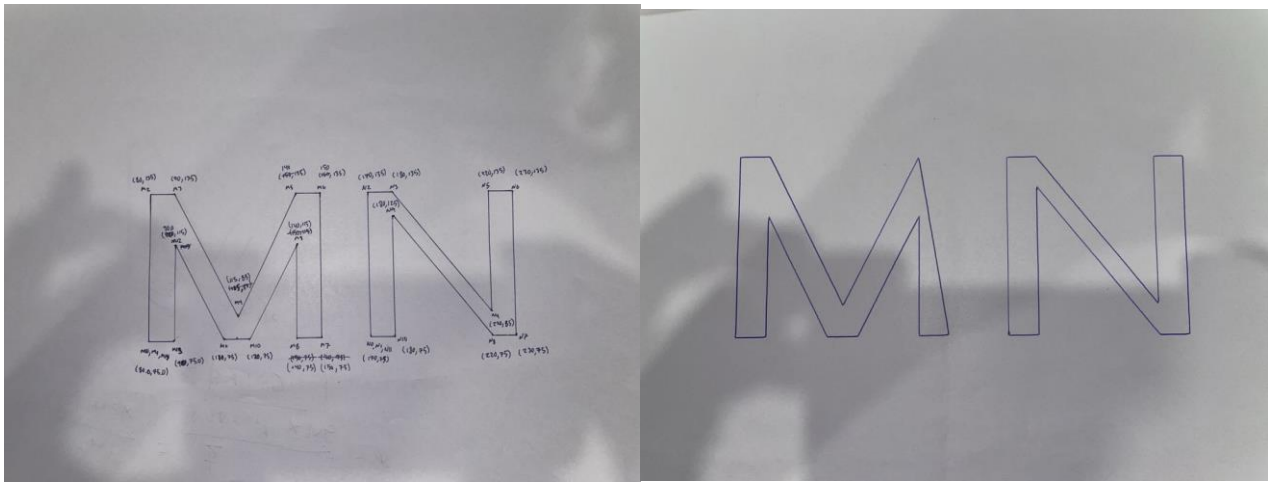


Figure 51: Letters MN with more complex geometry

4. References

“Technical Reference Manual RAPID Instructions, Functions and Data Types.” ABB.

“RMIT_EDUCATION CELLS Operating Manual.”